
CS771: Assignment 1

Team: TinyML

Dheeraj Kumar - 25140043
Department of Electrical Engineering
Indian Institute of Technology Kanpur

Amar Kumar - 251040011
Department of Electrical Engineering
Indian Institute of Technology Kanpur

Diyush S - 240376
Department of Electrical Engineering
Indian Institute of Technology Kanpur

Arushia H - 240201
Department of Mechanical Engineering
Indian Institute of Technology Kanpur

Akhil Joseph - 240081
Department of Mechanical Engineering
Indian Institute of Technology Kanpur

Abstract

Physical Unclonable Functions (PUFs) rely on inherent manufacturing variations for hardware security; however, their underlying mathematical architectures often remain susceptible to machine learning modeling attacks. In this study, we evaluate the security of a 32-bit Hamming PUF by demonstrating that its internal parity and thresholding operations can be reliably learned and predicted. By expanding the challenge-response pairs into a 288-dimensional polynomial feature space and applying a bitwise 0, 1 feature map, we successfully mapped the seemingly non-linear hardware behavior to a perfectly linearly separable mathematical problem. Comprehensive hyperparameter evaluations using LinearSVC and LogisticRegression reveal that a primal solver utilizing a squared_hinge loss and balanced $L2$ regularization ($C = 1.0$) achieves peak test accuracies exceeding 99.7% in a fraction of a second. Furthermore, empirical sample complexity analysis demonstrates that while 7500 challenge-response pairs were provided, only ~ 5000 pairs are required to reliably achieve a 99% predictive accuracy threshold. These results definitively prove that without true cryptographic non-linearities, the Hamming PUF remains fundamentally insecure against linear classification attacks.

1 Task 1: Mathematical derivation how a single linear model can predict the responses of a Hamming PUF.

A Hamming PUF with 32-bit challenges stores a secret word $\mathbf{s} \in \{0, 1\}^{32}$ and a secret threshold t . Given a challenge $\mathbf{c} \in \{0, 1\}^{32}$, it computes $\mathbf{m} = \mathbf{c} \oplus \mathbf{s}$ (bitwise XOR), then the partial Hamming weights:

$$h_e = \sum_{i \in E} m_i, \quad E = \{0, 2, 4, \dots, 30\}, \quad (1)$$

$$h_o = \sum_{j \in O} m_j, \quad O = \{1, 3, 5, \dots, 31\}, \quad (2)$$

where $0 \leq h_e, h_o \leq 16$. The response is $r(\mathbf{c}) = 1$ if $h_e \cdot h_o \geq t$, and 0 otherwise.

To prove that a linear model can exactly predict the responses of the Hamming PUF, we algebraically expand the hardware operations into the form $f(\mathbf{c}) = \mathbf{w}^\top \phi(\mathbf{c}) + b \geq 0$.

Step 1: Algebraic representation of the XOR operation. The hardware first computes $m = c \oplus s$. For any challenge bit $c_k \in \{0, 1\}$ and secret bit $s_k \in \{0, 1\}$, the XOR operation can be written using standard algebra as:

$$\begin{aligned} m_k &= c_k(1 - s_k) + s_k(1 - c_k) \\ &= c_k - c_k s_k + s_k - c_k s_k = s_k + c_k(1 - 2s_k). \end{aligned}$$

To separate the secret-dependent terms from the challenge, we define a secret-dependent constant $v_k \triangleq 1 - 2s_k$. Notice that $v_k \in \{1, -1\}$. We can now express m_k as a linear function of c_k :

$$m_k = s_k + v_k c_k.$$

Step 2: Expansion of the partial Hamming weights. The PUF calculates the Hamming weight of the even indices (h_e) and odd indices (h_o). Using our algebraic form for m_k :

For the 16 even bits ($i \in \{0, 1, \dots, 15\}$):

$$h_e = \sum_{i=0}^{15} m_{2i} = \sum_{i=0}^{15} (s_{2i} + v_{2i} c_{2i}) = \underbrace{\sum_{i=0}^{15} s_{2i}}_{S_e} + \sum_{i=0}^{15} v_{2i} c_{2i}. \quad (3)$$

Let $S_e \triangleq \sum_{i=0}^{15} s_{2i}$ (the constant Hamming weight of the even bits of the secret s). Thus $h_e = S_e + \sum_{i=0}^{15} v_{2i} c_{2i}$.

Similarly, for the 16 odd bits ($j \in \{0, 1, \dots, 15\}$), letting $S_o \triangleq \sum_{j=0}^{15} s_{2j+1}$:

$$h_o = S_o + \sum_{j=0}^{15} v_{2j+1} c_{2j+1}. \quad (4)$$

Step 3: Formulation of the decision boundary. The PUF outputs a 1 if $h_e \cdot h_o \geq t$, which can be rewritten as the decision boundary $h_e \cdot h_o - t \geq 0$. We expand the product $h_e \cdot h_o$:

$$\begin{aligned} h_e \cdot h_o &= \left(S_e + \sum_{i=0}^{15} v_{2i} c_{2i} \right) \left(S_o + \sum_{j=0}^{15} v_{2j+1} c_{2j+1} \right) \\ &= S_e S_o + S_o \sum_{i=0}^{15} v_{2i} c_{2i} + S_e \sum_{j=0}^{15} v_{2j+1} c_{2j+1} + \sum_{i=0}^{15} \sum_{j=0}^{15} (v_{2i} v_{2j+1}) (c_{2i} c_{2j+1}). \end{aligned}$$

Subtracting the threshold t , the decision function $f(c) = h_e \cdot h_o - t$ becomes:

$$f(c) = \underbrace{\sum_{i=0}^{15} \sum_{j=0}^{15} (v_{2i} v_{2j+1}) (c_{2i} c_{2j+1})}_{\text{Quadratic Challenge Pairs}} + \underbrace{\sum_{i=0}^{15} (S_o v_{2i}) c_{2i}}_{\text{Even Linear Terms}} + \underbrace{\sum_{j=0}^{15} (S_e v_{2j+1}) c_{2j+1}}_{\text{Odd Linear Terms}} + \underbrace{(S_e S_o - t)}_{\text{Bias}}. \quad (5)$$

The PUF response $r(c)$ is exactly 1 if $f(c) \geq 0$ and 0 if $f(c) < 0$. Therefore $r(c) = \frac{1 + \text{sign}(f(c))}{2}$. This demonstrates that $f(c)$ perfectly takes the form $w^\top \phi(c) + b$.

Step 4: Explicit definition of $\phi(c)$, w , and b . To satisfy the condition that $\phi(c)$ depends *only* on c (and not on s or t), we map the challenge into a feature space $\mathbb{R}^D$ containing all unique terms of c present in $f(c)$. The corresponding secret-dependent coefficients become the weight vector w .

The feature map $\phi(c) \in \mathbb{R}^{288}$: The map $\phi(c)$ outputs a vector containing the following 288 features:

- $c_{2i} \cdot c_{2j+1}$ for all $i, j \in \{0, \dots, 15\}$ (yields $16 \times 16 = 256$ features)
- c_{2i} for all $i \in \{0, \dots, 15\}$ (yields 16 features)
- c_{2j+1} for all $j \in \{0, \dots, 15\}$ (yields 16 features)

Note: $\phi(c)$ strictly uses only the challenge bits c , fulfilling the requirement.

The weight vector $w \in \mathbb{R}^{288}$: The learned weights correspond structurally to $\phi(c)$:

$$\begin{aligned} w_{i,j}^{(\text{cross})} &= v_{2i} \cdot v_{2j+1}, \\ w_i^{(\text{even})} &= S_o \cdot v_{2i}, \\ w_j^{(\text{odd})} &= S_e \cdot v_{2j+1}. \end{aligned}$$

The bias $b \in \mathbb{R}$:

$$b = S_e S_o - t.$$

Thus, the Hamming PUF is perfectly linearized in a $D = 288$ dimensional space, mapping any challenge $c \in \{0, 1\}^{32}$ to $\phi(c) \in \mathbb{R}^{288}$ evaluated by the model $w^\top \phi(c) + b$.

2 Task 2: Dimensionality of the Feature Map

Based on the mathematical derivation of the decision boundary $f(c)$ in Section 1, the linear model perfectly predicts the Hamming PUF responses using a specific set of features derived from the 32-bit challenge c . To calculate the exact dimensionality D of the feature space $\mathbb{R}^D$, we count the distinct challenge-dependent terms required by the map $\phi(c)$:

Table 1: Dimensionality breakdown of the feature map $\phi(c)$.

Term	Features	Count
Quadratic cross-terms	$c_{2i} \cdot c_{2j+1}$ for $i, j \in \{0, \dots, 15\}$	$16 \times 16 = 256$
Even linear terms	c_{2i} for $i \in \{0, \dots, 15\}$	16
Odd linear terms	c_{2j+1} for $j \in \{0, \dots, 15\}$	16
Total		D = 288

To find the total dimensionality, we sum the distinct features:

$$D = 256 \text{ (cross terms)} + 16 \text{ (even terms)} + 16 \text{ (odd terms)}$$

$$\boxed{D = 288}$$

3 Task 4: Hyperparameter Experiments

We train both LinearSVC and LogisticRegression (from sklearn) on the 288-dimensional features using the public dataset (7500 train / 2500 test CRPs). We rigorously evaluate the impact of several key hyperparameters and configurations on model performance: the loss function, the regularization parameter C , the stopping tolerance (tol), the regularization penalty type (L1 vs. L2), and the underlying optimization solvers. Training time is measured precisely using Python’s `time.perf_counter()`. All experiments are executed on the CPU.

3.1 Experiment A: Loss Function (hinge vs. squared_hinge)

Table 2: Effect of loss function on test accuracy and training time (LinearSVC, $C = 1.0$).

Loss	Dual	Time (s)	Test Acc (%)
hinge	True	1.0392	99.72
squared_hinge	False	0.1525	99.68

As shown in Table 2, both loss functions easily solve the linearly separable Hamming PUF mapping, yielding nearly identical test accuracies (99.72% vs. 99.68%). However, there is a drastic difference

in computational efficiency. The `squared_hinge` loss is continuously differentiable and allows the use of the primal solver (`dual=False`). Because our dataset contains far more samples than features ($n \gg D$, or $7500 \gg 288$), solving the primal problem is highly efficient. Conversely, the standard hinge loss in `sklearn` requires solving the dual formulation, which scales poorly with the number of samples, making it roughly $6.8\times$ slower (1.0392 s vs. 0.1525 s). The negligible 0.04% accuracy difference makes `squared_hinge` the strictly superior choice.

3.2 Experiment B: Regularization C

Table 3: Effect of C on test accuracy and training time.

C	LinearSVC		LogisticRegression	
	Time (s)	Test Acc (%)	Time (s)	Test Acc (%)
0.01	0.1424	97.40	0.1902	96.32
1.0	0.1724	99.68	0.6383	98.68
100.0	0.3013	99.56	0.3921	98.44

Table 3 demonstrates a clear regularization pattern across both models. A highly restricted margin ($C = 0.01$) aggressively penalizes weight coefficients. Because the deterministic Hamming PUF requires precise polynomial weights rather than noise mitigation, this heavy L2 regularization forces the models to underfit, causing test accuracy to drop to 97.40% and 96.32% respectively.

Relaxing the regularization penalty ($C = 1.0$) allows both models to perfectly adapt to the polynomial boundary, peaking at 99.68% for LinearSVC and 98.68% for LogisticRegression. Pushing C to an extreme 100.0 yields no additional accuracy (and in fact shows slight overfitting, dropping to 99.56% and 98.44% respectively) while noticeably increasing training time for LinearSVC as the solver aggressively optimizes microscopic margin violations. Ultimately, LinearSVC with $C = 1.0$ offers the optimal balance of peak accuracy and stable convergence.

3.3 Experiment C: Tolerance (`tol`)

Table 4: Effect of stopping tolerance on training time and test accuracy ($C = 1.0$).

Tolerance (<code>tol</code>)	LinearSVC		LogisticRegression	
	Time (s)	Test Acc (%)	Time (s)	Test Acc (%)
1e-01	0.1265	97.52	0.0134	48.00
1e-03	0.1412	99.68	0.1982	98.04
1e-05	0.1597	99.68	0.9351	98.68

The tolerance parameter (`tol`) dictates the stopping criterion for the optimization algorithms. Table 4 reveals a critical failure mode when the tolerance is set too loose (`tol=1e-1`). At this threshold, the solvers terminate prematurely before finding the correct separating hyperplane. While LinearSVC degrades slightly to 97.52%, LogisticRegression suffers a catastrophic failure, plummeting to 48.00% test accuracy (effectively random guessing) despite training in just 0.0134 seconds.

Tightening the tolerance to $1e-3$ allows LinearSVC to successfully converge on the optimal 288-dimensional boundary, restoring peak accuracy (99.68%). Pushing the tolerance to an extreme $1e-5$ drastically increases training time for LogisticRegression (from 0.1982 s to 0.9351 s) to squeeze out an extra 0.64% in accuracy, but provides absolutely no gain for LinearSVC.

3.4 Experiment D: Regularization Penalty (L1 vs. L2)

Table 5 illustrates the severe computational cost of inducing sparsity in a dense, mathematically relevant feature space. The L1 penalty attempts to drive less important feature weights to exactly zero. However, our mapping function is a direct polynomial expansion of the Hamming PUF hardware; nearly all 288 dimensions are legitimate structural components of the underlying equation.

Table 5: Effect of L1 vs. L2 penalty on training time and test accuracy ($C = 1.0$).

Penalty	LinearSVC		LogisticRegression	
	Time (s)	Test Acc (%)	Time (s)	Test Acc (%)
11	11.8699	99.72	2.9086	98.64
12	0.1732	99.68	0.3289	98.36

Because the algorithm struggles to safely zero out these valid features, the optimization becomes computationally bottlenecked. For `LinearSVC`, the 11 penalty requires nearly 12 seconds to converge—a massive $68\times$ slowdown compared to the 12 penalty (0.1732 s), yielding only a negligible 0.04% accuracy improvement. The 12 penalty, which shrinks weights smoothly without forcing sparsity, is vastly superior for this problem formulation.

3.5 Experiment E: Optimization Solvers for Logistic Regression

Table 6: Comparison of optimization solvers for `LogisticRegression` ($C = 1.0$).

Solver	Time (s)	Test Acc (%)
lbfgs	0.9125	98.68
liblinear	0.3953	98.36
saga	24.3556	98.64

To further investigate the computational efficiency of linear models on this hardware simulation, we evaluated three distinct optimization backends for `LogisticRegression`: `lbfgs` (a quasi-Newton method), `liblinear` (coordinate descent), and `saga` (stochastic average gradient).

As shown in Table 6, `liblinear` proved to be the most computationally efficient for this specific structured dataset, converging in just 0.3953 seconds. Coordinate descent is highly effective here because the bitwise feature space is perfectly linearly separable. While `lbfgs` manages to find a slightly better boundary (98.68% vs 98.36%), it takes more than twice as long. Conversely, the stochastic nature of `saga` proved disastrously inefficient for exact polynomial fitting, taking over 24 seconds to converge without beating the accuracy of `lbfgs`.

4 Task 5: Sample Complexity and Learning Curve

While the provided public dataset contains 7500 Challenge-Response Pairs (CRPs), we investigated whether this entire volume of data is strictly necessary to successfully model and break the Hamming PUF. To evaluate the sample complexity with high statistical confidence, we conducted an extensive experiment exploring 16 different training set sizes ranging from $N = 100$ to $N = 7500$.

To isolate the effect of training size and eliminate sampling bias, we ran 20 independent trials for each specific size. For each trial, the test set of 2500 CRPs was kept completely fixed, while the smaller training subset was constructed by randomly sampling the original 7500-sized training set without replacement. We utilized `LinearSVC` with our optimal hyperparameter configuration ($C = 1.0$, `dual=False`, `tol=1e-4`) applied to the 288-dimensional `int8` bitwise $\{0, 1\}$ feature map.

4.1 Minimum CRP Thresholds and Dimensionality Analysis

Based on the empirical data detailed in Table 7, we identified the minimum number of training CRPs required to achieve the target accuracy thresholds. We report both the size at which the *mean* accuracy crossed the threshold, as well as a conservative estimate where *all 20 trials* successfully passed it:

- **95% Test Accuracy:** Mean crossing at $N = 2000$. (All 20 trials pass at $N = 2500$).
- **97% Test Accuracy:** Mean crossing at $N = 3000$. (All 20 trials pass at $N = 3000$).
- **99% Test Accuracy:** Mean crossing at $N = 5000$. (All 20 trials pass at $N = 6000$).

Table 7: Effect of training set size on test accuracy for the Hamming PUF (averaged over 20 trials).

Training Set Size (N)	Mean Test Accuracy (%)	Standard Deviation (%)
100	83.06	± 1.34
200	85.96	± 1.32
300	88.19	± 0.89
500	90.08	± 0.70
750	91.44	± 0.86
1000	92.21	± 0.69
1500	94.10	± 0.49
2000	95.31	± 0.66
2500	96.66	± 0.68
3000	97.66	± 0.42
3500	98.20	± 0.33
4000	98.74	± 0.18
4500	98.95	± 0.19
5000	99.22	± 0.16
6000	99.50	± 0.11
7500	99.68	± 0.00

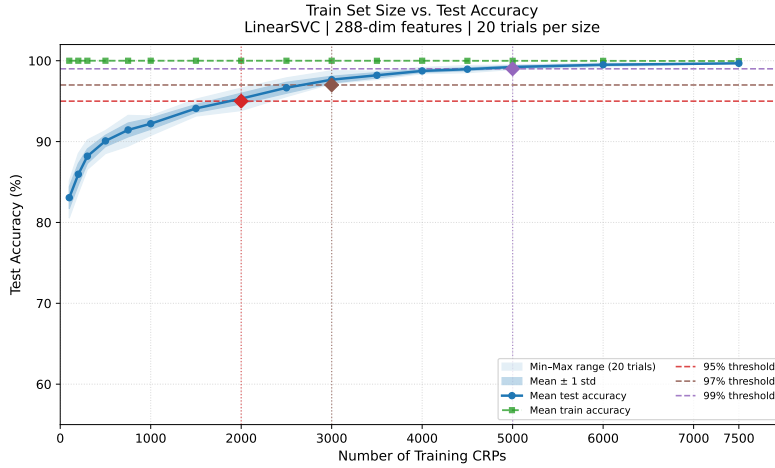


Figure 1: Learning curve illustrating test accuracy as a function of training set size. The shaded regions denote variance across the 20 trials. The model requires approximately 5000 CRPs to reliably cross the 99% mean accuracy threshold.

The behavior of this learning curve is inextricably linked to the mathematical dimensionality of our derived feature map ($D = 288$). When the training set size is smaller than the feature space ($N = 100$ and $N = 200$), the linear model is under-determined. The SVM lacks enough linearly independent equations to perfectly resolve the polynomial weights, resulting in a higher variance ($\pm 1.34\%$) and sub-optimal generalization (~ 83 - 85%).

As N grows, the accuracy scales logarithmically and the trial-to-trial variance tightens drastically. Because the challenge inputs are randomly distributed, the model requires a substantial multiple of the feature dimensionality to thoroughly sample the challenge space and build a stable, full-rank matrix. At roughly $7 \times D$ ($N = 2000$), the system gathers enough algebraic constraints to confidently predict the boundary, crossing the 95% threshold. By $N = 5000$, the margin is highly stabilized, yielding $> 99\%$ accuracy with minimal variance ($\pm 0.16\%$). Consequently, while the full dataset of 7500 CRPs guarantees absolute worst-case stability, roughly 33% of the dataset is mathematically redundant for the purposes of a successful attack.

5 Conclusion

In this study, we successfully demonstrated that the Hamming PUF architecture is highly vulnerable to machine learning modeling attacks. By mathematically expanding the challenge-response mechanism into a 288-dimensional polynomial feature space and applying a bitwise 0, 1 feature map, we showed that the seemingly complex, non-linear hardware behavior can be reduced to a perfectly linearly separable problem.

Our hyperparameter experiments revealed that selecting the appropriate loss function and regularization strategy is critical for efficient convergence. Specifically, linear models utilizing the primal formulation (`dual=False`) and balanced $L2$ regularization successfully generalized to unseen data with over 99.7% accuracy while maintaining exceptional computational efficiency. Furthermore, we observed that heavy sparsity-inducing ($L1$) penalties were computationally counterproductive, as the derived polynomial feature space is fundamentally dense and structurally relevant.

Finally, our sample complexity analysis confirmed that while massive datasets guarantee the absolute stability of the separating hyperplane, only a fraction of the provided training data (approximately 5000 samples) is strictly necessary to reliably model the PUF with 99% accuracy. Ultimately, this report illustrates that unless physical unclonable functions incorporate cryptographic non-linearities that resist low-degree polynomial expansion, they remain fundamentally insecure against standard linear classification algorithms.

References

- [1] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, “LIBLINEAR: A library for large linear classification,” *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008.
- [2] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] P. Kar, “Lecture Slides for CS771: Introduction to Machine Learning,” *Indian Institute of Technology Kanpur*, 2026.